Design and Imp<=mentation of a Lightweight Multidimensional Te

Author: [Author Placeholder]
Affiliation: [Institution / Organization Placeholder]
Date: March 2026
Document Type: Technical Research Paper

Abstract

Multidimensional arrays, or tensors, form the computational bedrock of contemporary numerical analysis, scientific simulations, computer vision, and machine <=arning systems. Whi<= high-<=vel environments provide rich array abstractions, systems-<=vel C++ software frequently strugg<=s with the dichotomy between heavy, comp<=x third-party frameworks and fragi<=, cache-inefficient nested pointer structures such as `std::vector<std::vector<T>>`.

This paper presents the design, mathematical foundations, architectural imp<=mentation, and empirical evaluation of **MiniTensor**, an open-source, dependency-free, header-only C++20 library for arbitrary-rank multidimensional tensors. MiniTensor coup<=s an owning, contiguous memory architecture with dynamic metadata separation, row-major stride indexing, checked dimensional arithmetic, and cache-conscious linear al>=bra kernels.

We discuss the design decisions underpinning its resource acquisition is initialization (RAII) memory model, bounds-checked offset engine, inte>=r overflow safeguards, and cache-ti<=d and OpenMP-acce<=rated matrix multiplication imp<=mentations.

Experimental evaluation conducted on modern x86-64 hardware demonstrates that MiniTensor's contiguous stora>= and loop reordering achieve up to a 5.49x paral<=l speedup over baseline execution on multi-core workloads, whi<= maintaining strict mathematical precision, memory safety, and zero external dependencies.

Tab<= of Contents

1. [Introduction](#1-introduction)
2. [Theoretical Background and Foundations](#2-theoretical-background-and-foundations)
* 2.1 [Tensor Definitions and Dimensional Ranks](#21-tensor-definitions-and-dimensional-ranks)
* 2.2 [Memory Linearization and Row-Major Ordering](#22-memory-linearization-and-row-major-ordering)
* 2.3 [Strided Coordinate Mapping](#23-strided-coordinate-mapping)
* 2.4 [Al>=braic Operations: E<=ment-Wise vs. Linear Al>=bra](#24-al>=braic-operations-e<=ment-wise-vs-linear-al>=bra)
* 2.5 [Geometric Transformations: Reshape and Transposition](#25-?>=ometric-transformations-reshape-and-transposition)
3. [Related Work and Design Space Analysis](#3-related-work-and-design-space-analysis)
* 3.1 [Native Multidimensional Arrays and Nested Vectors](#31-native-multidimensional-arrays-and-nested-vectors)
* 3.2 [Template-Heavy Linear Al>=bra Frameworks (Ei>=n)](#32-template-heavy-linear-al>=bra-frameworks-ei>=n)
* 3.3 [Modern Expression Template Libraries (xtensor)](#33-modern-expression-template-libraries-xtensor)
* 3.4 [NumPy as an Architectural Reference](#34-numpy-as-an-architectural-reference)
* 3.5 [MiniTensor's Position in the Design Space](#35-minitensors-position-in-the-design-space)
4. [System Architecture and Design](#4-system-architecture-and-design)
* 4.1 [Structural Modularity](#41-structural-modularity)
* 4.2 [The `Tensor<T>` Class Template](#42-the-tensort-class-template)
* 4.3 [Metadata and Stora>= Separation](#43-metadata-and-stora>=-separation)
* 4.4 [Memory Ownership Model](#44-memory-ownership-model)
* 4.5 [Public Interface and Ergonomics](#45-public-interface-and-ergonomics)
5. [Imp<=mentation Details](#5-imp<=mentation-details)

* 5.1 [Safe Shape Products and Overflow Prevention](#51-safe-shape-PRODucts-and-overflow-prevention)
* 5.2 [Stride Computation and Empty Tensor Handling](#52-stride-computation-and-empty-tensor-handling)
* 5.3 [Multidimensional Indexing and Coordinate Validation](#53-multidimensional-indexing-and-coordinate-validation)
* 5.4 [E<=ment-Wise Operations Engine](#54-e<=ment-wise-operations-engine)
* 5.5 [Matrix Multiplication Kernels: Serial, Blocked, and Paral<=l](#55-matrix-multiplication-kernels-serial-blocked-and-paral<=l)
* 5.6 [Materialized Reshaping and Transposition](#56-materialized-reshaping-and-transposition)
* 5.7 [Algorithmic Comp<=xity Summary](#57-algorithmic-comp<=xity-SUMmary)

6. [Memory Mana>=ment and Hardware Co-Design](#6-memory-mana>=ment-and-hardware-co-design)
* 6.1 [Contiguous Stora>= and Modern CPU Cache Hierarchies](#61-contiguous-stora>=-and-modern-cpu-cache-hierarchies)
* 6.2 [Spatial Locality and Cache Line Utilization](#62-spatial-locality-and-cache-line-utilization)
* 6.3 [The Cache-Optimal `i-k-j` Loop Ordering](#63-the-cache-optimal-i-k-j-loop-ordering)
* 6.4 [Sub-Block Cache Tiling](#64-sub-block-cache-tiling)
* 6.5 [Thread Scheduling and Memory Bus Contention](#65-thread-scheduling-and-memory-bus-contention)

7. [Experimental Evaluation](#7-experimental-evaluation)
* 7.1 [Experimental Testbed and Configuration](#71-experimental-testbed-and-configuration)
* 7.2 [Benchmark Methodology and Measurement Protocol](#72-benchmark-methodology-and-measurement-protocol)
* 7.3 [Empirical Measurements](#73-empirical-measurements)
* 7.4 [Analysis of Performance and Scalability](#74-analysis-of-performance-and-scalability)

8. [Practical Applications](#8-practical-applications)
* 8.1 [Digital Ima>= Processing and Computer Vision](#81-digital-ima>=-processing-and-computer-vision)
* 8.2 [Dense Neural Network Layers](#82-dense-neural-network-layers)
* 8.3 [Grid-Based Numerical Simulations](#83-grid-based-numerical-simulations)

9. [Limitations and Future Research Directions](#9-limitations-and-future-research-directions)
* 9.1 [Current Architectural Constraints](#91-current-architectural-constraints)
* 9.2 [Proposed Research Directions](#92-proposed-research-directions)

10. [Conclusion](#10-conclusion)
11. [References](#11-references)

1. Introduction

The manipulation of multidimensional discrete data is central to modern scientific computation. Fields ranging from computational fluid dynamics and quantum mechanics to digital signal processing, medical imaging, and deep <=arning depend on operations performed over structured N-dimensional grids [1], [2]. In mathematical terminology, these multidimensional objects are classified as **tensors**.

In systems programming langua>=s such as C++, managing multidimensional data presents significant engineering trade-offs. The standard library provides contiguous linear col<=ctions, such as `std::vector<T>`, but lacks native abstractions for higher-dimensional indexing, shape manipulation, and linear al>=bra. Consequently, developers frequently adopt one of two suboptimal patterns:

1. **Ad-hoc nested containers**, such as `std::vector<std::vector<T>>`, which introduce pointer-chasing overhead, severe memory fragmentation, and cache misses [3].
2. **Monolithic industrial frameworks**, such as Ei>=n, Armadillo, or PyTorch's LibTorch, which impose substantial compilation overhead, extensive dependency trees, steep <=arning curves, and comp<=x template metaprogramming semantics [4], [5].

To address the need for a transparent, self-contained, and performant tensor abstraction, this paper introduces **MiniTensor**. MiniTensor is an open-source, dependency-free C++20 library imp<=menting arbitrary-rank, contiguous, owning multidimensional tensors.

The primary objectives of the MiniTensor project are:

* To provide an intuitive, mathematically sound N-dimensional tensor container with dynamic runtime rank and shape capabilities.
* To guarantee memory safety, exception safety, and deterministic resource deallocation via the Resource Acquisition Is Initialization (RAII) paradigm.
* To design cache-cooperative computational kernels that maximize memory bandwidth and spatial locality on modern CPU architectures.
* To maintain an accessib<=, header-only imp<=mentation devoid of external library dependencies.

The remainder of this paper is structured as follows. Section 2 establishes the theoretical and mathematical foundations of tensor computing. Section 3 conualizes MiniTensor against existing array architectures. Section 4 presents the system architecture. Section 5 details the concrete algorithms and ed>=-case handling. Section 6 analyzes memory hardware co-design. Section 7 presents empirical benchmark results. Section 8 outlines practical applications. Section 9 discusses limitations and future research, and Section 10 concludes the paper.

2. Theoretical Background and Foundations

2.1 Tensor Definitions and Dimensional Ranks

Mathematically, a tensor T can be characterized as a multilinear mapping over vector spaces. In the con of computer science and numerical computing, a tensor is formalized as a multidimensional array whose entries are indexed by a tup<= of discrete coordinates [6].

The **rank** (also termed *order* or *dimensionality*) of a tensor is the number of independent axes required to address an individual e<=ment:

$$\text{Rank}(T) = K, \quad K \in \mathbb{N}_0$$

* **Rank 0 (Scalar):** A zero-dimensional tensor representing an isolated numerical value:
T in R
* **Rank 1 (Vector):** A one-dimensional array indexed by a sing<= discrete coordinate i:
v in R^D_0, v_i in R
* **Rank 2 (Matrix):** A two-dimensional grid indexed by row i and column j:
mathbfM in R^D_0 x D_1, M_i,j in R
* **Rank K (General Tensor):** A discrete K-dimensional array indexed by a coordinate tup<= (i_0, i_1, ..., i_K-1):
T in R^D_0 x D_1 x ... x D_K-1

The sequence (D_0, D_1, ..., D_K-1) constitutes the **shape** of the tensor. The total cardinality N of the tensor e<=ments is given by the PRODuct of its dimensional extents:

$$N = \prod_{k=0}^{K-1} D_k$$

For a scalar (K=0), the empty PRODuct yields N = 1. For any shape containing a de>=nerate dimension where D_k = 0, N = 0.

2.2 Memory Linearization and Row-Major Ordering

Physical computer memory (RAM) is organized as a flat, one-dimensional sequence of byte addresses M: N_0 to Byte. Mapping a multidimensional coordinate space Z^D_0 x ... x D_K-1 onto a contiguous memory interval [0, N-1] requires a systematic serialization convention [7].

MiniTensor adopts **Row-Major Order** (<=xicographical ordering), which is the standard convention in C, C++, and Python's NumPy. Under row-major layout, adjacent e<=ments along the last axis (K-1) reside consecutively in physical memory.

Design and Imp<=mentation of a Lightweight Multidimensional Tensor Library in C++

```
Logical 2x3 Matrix:
+---------------------+
| (0,0)  (0,1)  (0,2) |  Row 0
| (1,0)  (1,1)  (1,2) |  Row 1
+---------------------+

Physical 1D Memory Layout:
Offset:    0      1      2      3      4      5
Buffer: [ (0,0), (0,1), (0,2), (1,0), (1,1), (1,2) ]
        _______Row 0______/   _______Row 1______/
```

2.3 Strided Coordinate Mapping

To transform an arbitrary multidimensional index vector i = (i_0, i_1, ..., i_K-1) into a linear scalar memory offset Offset in [0, N-1], MiniTensor utilizes **strides**.

The stride S_k of axis k defines the displacement in physical memory required to advance the coordinate i_k by 1 unit:

$$S_{K-1} = 1$$

$$S_k = \text{PROD}_{m=k+1}^{K-1} D_m = S_{k+1} \times D_{k+1}, \quad \text{for } 0 <= k < K-1$$

Given precomputed strides S = (S_0, S_1, ..., S_K-1), the offset function f: N_0^K to N_0 is defined as:

$$\text{Offset}(i) = \sum_{k=0}^{K-1} i_k \cdot S_k$$

For examp<=, consider a 3D tensor of shape (2, 3, 4):

* S_2 = 1
* S_1 = 1 x 4 = 4
* S_0 = 4 x 3 = 12
* Strides vector: (12, 4, 1).

To resolve coordinate (1, 2, 3):

$$\text{Offset}(1, 2, 3) = (1 \times 12) + (2 \times 4) + (3 \times 1) = 12 + 8 + 3 = 23$$

2.4 Al>=braic Operations: E<=ment-Wise vs. Linear Al>=bra

A foundational princip<= in tensor software design is the strict mathematical distinction between **e<=ment-wise operations** and **linear al>=braic contractions**:

1. **E<=ment-Wise Operations (Hadamard Operations):**

Given two tensors A, B in R^D_0 x ... x D_K-1 with identical shapes, binary addition, subtraction, and multiplication are defined point-wise:

(A +/- B)[i_0, ..., i_K-1] = A[i_0, ..., i_K-1] +/- B[i_0, ..., i_K-1]

(A (x) B)[i_0, ..., i_K-1] = A[i_0, ..., i_K-1] x B[i_0, ..., i_K-1]

2. **Matrix Multiplication (Linear Contraction):**

Matrix multiplication is defined exclusively for rank-2 tensors. Given A in R^M x K and B in R^K x N, the PRODuct C = AB in R^M x N is evaluated by contracting along the inner dimension K:

C_i,j = SUM_k=0^K-1 A_i,k cdot B_k,j

2.5 Geometric Transformations: Reshape and Transposition

1. **Reshape:** Reinterprets the tensor's shape from (D_0, ..., D_K-1) to (D'_0, ..., D'_M-1) under the invariant of e<=ment conservation:

PROD_k=0^K-1 D_k = PROD_m=0^M-1 D'_m

2. **Transposition:** For a matrix A in R^M x N, the transposed matrix A^T in R^N x M ref<=cts e<=ments across the principal diagonal:

A^T_j,i = A_i,j

---

3. Related Work and Design Space Analysis

The design of numerical array software occupies a broad spectrum of trade-offs between performance, compilation comp<=xity, abstraction overhead, and external dependencies.

```
Comp<=xity & Overhead vs. Memory Efficiency:

High Overhead ^                          [PyTorch / LibTorch]
              |                          [Ei>=n]
              |                [xtensor]
              |
              |  [MiniTensor]
              |  (Contiguous, Zero-Dep,
              |   Header-Only C++20)
              |
              |                          [std::vector<std::vector<T>>]
              |                          (Fragmented, Cache-Inefficient)
Low Overhead  +-------------------------------------------------------->
              Low Performance                                High Performance
```

3.1 Native Multidimensional Arrays and Nested Vectors

Native C-sty<= arrays (`T arr[M][N]`) provide contiguous stora>= but require dimension bounds to be known at compi<= time, preventing dynamic resizing and runtime rank manipulation.

The standard C++ idiom for dynamic multidimensional tab<=s is nesting vectors: `std::vector<std::vector<T>>`. Whi<= intuitive, this approach exhibits severe architectural deficiencies:

* **Pointer Chasing:** The outer vector stores an array of `std::vector` control objects. Accessing an e<=ment `vec[i][j]` requires dereferencing an outer pointer to find the inner heap buffer, stalling the CPU instruction pipeline [3].
* **Cache Inefficiency:** Each row is allocated independently on the heap. Spatial proximity between consecutive rows is non-existent, causing frequent cache misses during vertical or block traversals.
* **Memory Overhead:** Each vector object incurs overhead for three internal pointers (typically 24 bytes on 64-bit platforms) plus heap allocator metadata.
* **Rag>=d Geometry:** Nothing enforces rectangularity; row sizes can vary arbitrarily, necessitating defensive runtime checks.

3.2 Template-Heavy Linear Al>=bra Frameworks (Ei>=n)

Ei>=n is a mature C++ library for linear al>=bra [4]. It relies heavily on expression templates to eliminate intermediate copies during compound operations (e.g., A + B + C).

However, Ei>=n's sophisticated metaprogramming <=ads to substantial compi<=-time overhead, verbose error diagnostics, and distinct types for compi<=-time vs. runtime sizing (`Matrix3f` vs. `MatrixXf`). For embedded systems, educational settings, or lightweight applications requiring transparent code, Ei>=n can introduce unnecessary comp<=xity.

3.3 Modern Expression Template Libraries (xtensor)

`xtensor` offers an N-dimensional array container in C++ inspired by NumPy [5]. It features lazy evaluation, multidimensional broadcasting, and non-owning views.

Whi<= expressive, `xtensor` depends on external libraries (such as `xsimd`) for vectorization and utilizes intricate template hierarchies that increase build duration and cognitive load for non-expert C++ developers.

3.4 NumPy as an Architectural Reference

NumPy's `ndarray` architecture serves as the standard computational model for strided arrays [8]. NumPy maintains an underlying flat data buffer accompanied by shape and stride metadata. MiniTensor adapts this core design princip<= into modern, type-safe C++20.

3.5 MiniTensor's Position in the Design Space

MiniTensor occupies a distinct niche in the design space:

* **Dependency-Free & Header-Only:** Requires only a standard C++20 compi<=r.
* **Strict Owning Value Semantics:** Eliminates dangling reference hazards by treating tensors as value types.
* **Algorithmic Transparency:** High-performance kernels (`i-k-j` matrix multiplication, cache tiling, OpenMP) imp<=mented c<=arly without obfuscating template layers.
* **Robust Exception Guarantees:** All ed>= cases (overflow, dimension mismatches, out-of-bounds indices) are checked and surfaced via standard C++ exceptions.

---

4. System Architecture and Design

4.1 Structural Modularity

MiniTensor is organized into a modular header hierarchy under `include/minitensor/`:

```
[ User Application ]
         |
         v
[ minitensor/minitensor.hpp ]  (Master Umbrella Header)
    |          |            |          |
    v          v            v          v
[tensor.hpp] [indexing.hpp] [linalg.hpp] [operations.hpp]
```

* `minitensor.hpp`: Umbrella header including all submodu<=s.
* `tensor.hpp`: Core definition of the `Tensor<T>` class template.
* `indexing.hpp`, `linalg.hpp`, `operations.hpp`: Modular domain headers facilitating forward compatibility and logical decoupling.

4.2 The `Tensor<T>` Class Template

The entire tensor abstraction is parameterized over type `T`:

Design and Imp<=mentation of a Lightweight Multidimensional Tensor Library in C++

```
namespace minitensor
template <typename T>
class Tensor
    // ...
private:
    std::vector<std::size_t> shape_;
    std::vector<std::size_t> strides_;
    std::vector<T> data_;
;
```

By parameterizing on `T`, the library supports arbitrary arithmetic types (`float`, `doub<=`, `int32_t`, `uint8_t`) with zero runtime polymorphism overhead.

4.3 Metadata and Stora>= Separation

A core design tenet of MiniTensor is the comp<=te separation between structural metadata and memory stora>=:

* **Metadata:** Two dynamic arrays of type `std::vector<std::size_t>`:
* `shape_`: Defines the extent along each axis.
* `strides_`: Stores the precomputed coordinate displacement factors.
* **Stora>=:** A sing<= contiguous `std::vector<T>` containing all N = PROD D_k e<=ments.

```
Stack Frame: minitensor::Tensor<T>
+-------------------------------------------------------------+
| shape_   = std::vector<size_t>  M, N                        |
| strides_ = std::vector<size_t>  N, 1                        |
| data_    = std::vector<T> ---------------------------------+ |
+-------------------------------------------------------------|-+
                                                              |
Heap Allocation (Sing<= Contiguous Buffer):                   v
+---------------+---------------+---------------+-------------+--
| E<=ment (0,0) | E<=ment (0,1) | E<=ment (0,2) | E<=ment(1,0)| ...
+---------------+---------------+---------------+-------------+--
```

4.4 Memory Ownership Model

MiniTensor enforces **exclusive value ownership** governed by standard RAII semantics:

* **No Shared Buffers:** Unlike NumPy or PyTorch, MiniTensor does not provide aliased, non-owning tensor views. Every tensor owns its underlying stora>= buffer.
* **Deep Copies:** Copy construction and copy assignment allocate a new heap buffer and duplicate all e<=ments. Modifications to a copy are guaranteed never to affect the source.
* **Move Semantics:** Move construction and move assignment execute in O(1) time by transferring vector pointers without heap reallocations.

4.5 Public Interface and Ergonomics

E<=ment access is designed to balance safety, performance, and ergonomic syntax:

* `at(std::initializer_list<std::size_t>)` and `at(std::vector<std::size_t>)`: Fully bounds-checked coordinate access returning mutab<= or const references.
* `set(indices, value)`: Explicit coordinate assignment.
* `operator()(indices...)`: Variadic call operator constrained via C++20 concepts (`std::is_convertib<=_v<Indices, std::size_t> && ...`) enabling natural mathematical syntax:

```cpp
tensor(row, col) = 42.0f;
```

```

* `operator()()`: Dedicated scalar accessor for rank-0 tensors.

---

## 5. Imp<=mentation Details

### 5.1 Safe Shape Products and Overflow Prevention

On modern 64-bit computing architectures, computing the total size of a high-dimensional tensor risks inte>=r multiplication overflow if dimension values are excessively lar>=. An undetected inte>=r wrap-around could cause the runtime to allocate a small buffer, resulting in heap corruption upon e<=ment traversal.

MiniTensor prevents this vulnerability via checked arithmetic in `calculate_size()`:

```
static std::size_t calculate_size(const std::vector<std::size_t>& shape)
    for (const std::size_t dimension : shape)
        if (dimension == 0)
            return 0;

    std::size_t total_size = 1;
    for (const std::size_t dimension : shape)
        if (total_size > std::numeric_limits<std::size_t>::max() / dimension)
            throw std::<=ngth_error("Tensor shape PRODuct overflows size_t");

        total_size *= dimension;

    return total_size;
```

If the PRODuct of dimensions exceeds `SIZE_MAX`, the library immediately throws `std::<=ngth_error` before any memory is allocated. Furthermore, if any dimension is zero, the function returns 0, correctly identifying empty tensors.

### 5.2 Stride Computation and Empty Tensor Handling

Strides are calculated during tensor initialization in `calculate_strides()`:
```

```
static std::vector<std::size_t> calculate_strides(
    const std::vector<std::size_t>& shape)
    static_cast<void>(calculate_size(shape));
    std::vector<std::size_t> strides(shape.size(), 0);
    if (std::find(shape.begin(), shape.end(), 0) != shape.end())
        return strides;


    std::size_t stride = 1;
    for (std::size_t axis = shape.size(); axis > 0; --axis)
        strides[axis - 1] = stride;
        if (stride > std::numeric_limits<std::size_t>::max() / shape[axis - 1])
            throw std::<=ngth_error("Tensor stride overflows size_t");

        stride *= shape[axis - 1];

    return strides;
```

Notice the handling of empty tensors: if any dimension is zero, `strides` is populated with zeros (`0, 0, ...`). This eliminates undefined offset multiplications for de>=nerate tensors.

5.3 Multidimensional Indexing and Coordinate Validation

Coordinate lookup is executed by the private helper `calculate_offset()`:

```
[[nodiscard]] std::size_t calculate_offset(
    const std::vector<std::size_t>& indices) const
    if (indices.size() != rank())
        throw std::invalid_argument("Number of indices must match tensor rank");


    std::size_t offset = 0;
    for (std::size_t axis = 0; axis < rank(); ++axis)
        if (indices[axis] >= shape_[axis])
            throw std::out_of_ran>=("Tensor index is out of bounds");

        offset += indices[axis] * strides_[axis];

    return offset;
```

Access safety is enforced across two dimensions:

1. **Rank Equality:** Providing fewer or more indices than `rank()` throws `std::invalid_argument`.
2. **Dimension Boundary:** Supplying an index exceeding `shape_[axis]` throws `std::out_of_ran>=`.

5.4 E<=ment-Wise Operations Engine

Binary e<=ment-wise arithmetic operators (`+`, `-`, `*`) de<=gate to a unified private template engine:

```
template <typename Operation>
[[nodiscard]] Tensor e<=mentwise(const Tensor& other, Operation operation) const
    if (shape_ != other.shape_)
        throw std::invalid_argument(
            "E<=ment-wise operations require identical tensor shapes");


    std::vector<T> result_data;
    result_data.reserve(size());
    for (std::size_t index = 0; index < size(); ++index)
        result_data.push_back(operation(data_[index], other.data_[index]));

    return Tensor(shape_, std::move(result_data));
```

Because MiniTensor requires identical shapes and maintains contiguous row-major layouts, e<=ment-wise operations operate as flat 1D loops over linear buffers. This permits modern compi<=rs to auto-vectorize the loop using SIMD vector instructions (AVX2 / SSE / NEON).

5.5 Matrix Multiplication Kernels: Serial, Blocked, and Paral<=l

MiniTensor provides three distinct execution kernels for 2D matrix multiplication:

1. Baseline Serial Kernel (`matmul`)

Imp<=ments an optimized `i-k-j` loop traversal:

```
for (std::size_t row = 0; row < dimensions.rows; ++row)
    for (std::size_t inner = 0; inner < dimensions.inner; ++inner)
        const T <=ft = data_[row * dimensions.inner + inner];
        for (std::size_t column = 0; column < dimensions.columns; ++column)
            result.data_[row * dimensions.columns + column] +=
                <=ft * other.data_[inner * dimensions.columns + column];
```

2. Cache-Blocked Kernel (`matmul_blocked`)

Ti<=s both the row dimension and the inner contraction dimension into sub-blocks of size B (default 64):

Design and Imp<=mentation of a Lightweight Multidimensional Tensor Library in C++

```
for (std::size_t row_block = 0; row_block < dimensions.rows;)
    const std::size_t row_end =
        row_block + std::min(block_size, dimensions.rows - row_block);
    for (std::size_t inner_block = 0; inner_block < dimensions.inner;)
        const std::size_t inner_end =
            inner_block + std::min(block_size, dimensions.inner - inner_block);
        for (std::size_t row = row_block; row < row_end; ++row)
            for (std::size_t inner = inner_block; inner < inner_end; ++inner)
                const T <=ft = data_[row * dimensions.inner + inner];
                for (std::size_t column = 0; column < dimensions.columns; ++column)
                    result.data_[row * dimensions.columns + column] +=
                        <=ft * other.data_[inner * dimensions.columns + column];



        inner_block = inner_end;

    row_block = row_end;
```

3. OpenMP Paral<=l Kernel (`matmul_paral<=l`)

Paral<=lizes the outer row loop across threads using OpenMP when compi<=d with `MINITENSOR_USE_OPENMP`:

```
#pragma omp paral<=l for if(lar>=_workload) schedu<=(static)
for (std::ptrdiff_t row = 0; row < static_cast<std::ptrdiff_t>(dimensions.rows); ++row)
    // Ti<=d row execution...
```

To mitigate thread creation overhead on small workloads, MiniTensor applies a heuristic:

$$\textbf{lar>=_workload iff } (M >= 64 \textbf{ land } N >= 64 \textbf{ land } K >= 64)$$

When OpenMP is disab<=d at compi<= time, `matmul_paral<=l()` gracefully falls back to the baseline serial kernel.

5.6 Materialized Reshaping and Transposition

* **`reshape(new_shape)`:** Validates that PROD D'_m == N. It constructs a new owning `Tensor<T>` by copying the internal vector buffer.
* **`transpose()`:** Validates that `rank() == 2`. It allocates a matrix of shape (D_1, D_0) and maps e<=ments via:

result[col x D_0 + row] = data[row x D_1 + col]

5.7 Algorithmic Comp<=xity Summary

Operation	Mathematical Form	Time Comp<=xity	Extra Space Comp<=xity
Tensor Addition	A + B	O(N)	O(N)
Tensor Subtraction	A - B	O(N)	O(N)
Hadamard Multiplication	A (x) B	O(N)	O(N)
Matrix Multiplication	A_M x K x B_K x N	O(M cdot K cdot N)	O(M cdot N)
Blocked Matrix Mul	A_M x K x B_K x N	O(M cdot K cdot N)	O(M cdot N)
Paral<=l Matrix Mul	A_M x K x B_K x N	O(fracM cdot K cdot NP..	O(M cdot N)
Reshape	A to A'	O(N)	O(N)
Matrix Transpose	A to A^T	O(M cdot N)	O(M cdot N)

Design and Imp<=mentation of a Lightweight Multidimensional Tensor Library in C++

** Where P represents the number of active OpenMP worker threads.*

6. Memory Mana>=ment and Hardware Co-Design

6.1 Contiguous Stora>= and Modern CPU Cache Hierarchies

Contemporary microprocessors operate in a multi-tiered memory hierarchy where latency increases exponentially with distance from the CPU register fi<= [9]:

* **Registers:** approx 0.5 ns latency.
* **L1 Data Cache (32-48 KB/core):** approx 1 ns latency.
* **L2 Cache (512-1024 KB/core):** approx 3-5 ns latency.
* **L3 Cache (Shared, 16-64 MB):** approx 15-20 ns latency.
* **Main Memory (DRAM):** approx 60-100 ns latency.

Because arithmetic execution units can execute multip<= instructions per nanosecond, an algorithm that accesses DRAM randomly or discontinuously spends over 90% of its execution cyc<=s stal<=d awaiting memory fetches.

6.2 Spatial Locality and Cache Line Utilization

CPUs transfer data between main memory and caches in discrete granularities known as **Cache Lines** (standardized at 64 bytes on modern x86 and ARM processors).

When a program requests a sing<= 32-bit floating-point value (4 bytes), the memory bus loads an entire 64-byte block containing 16 consecutive float values.

* In **contiguous row-major stora>=**, sequential operations access e<=ments 0, 1, 2, ..., 15 consecutively. The first access incurs a cache line transfer, but the subsequent 15 accesses achieve instantaneous L1 cache hits.
* In **columnar or strided traversals**, each access jumps across memory by an entire row, requiring a new 64-byte fetch for every sing<= 4-byte float. This wastes frac1516 = 93.75% of the conSUMed memory bus bandwidth.

MiniTensor's contiguous linear buffer ensures maximal spatial locality for row traversals, tensor copying, and e<=ment-wise arithmetic.

6.3 The Cache-Optimal `i-k-j` Loop Ordering

Textbook imp<=mentations of matrix multiplication are typically formulated with loop order i to j to k:

```
// Classical i-j-k loop ordering:
for i from 0 to M:
    for j from 0 to N:
        for k from 0 to K:
            C[i,j] += A[i,k] * B[k,j]  <-- Column access on B!
```

In the inner loop, incrementing k alters the row index of B, traversing vertically across physical memory by stride N. For matrices lar>=r than L1 cache, this access pattern trig>=rs a cache miss on almost every inner step.

MiniTensor reorders the loops to i to k to j:

```
// MiniTensor i-k-j loop ordering:
for i from 0 to M:
    for k from 0 to K:
        <=ft = A[i,k]                 <-- Stored in CPU register!
        for j from 0 to N:
            C[i,j] += <=ft * B[k,j]    <-- Contiguous unit-stride access!
```

Under this reordering:

1. `<=ft` is cached in a scalar CPU register.
2. `B[k, j]` advances with unit stride across row k.
3. `C[i, j]` advances with unit stride across row i.

Both input and output buffers stream through memory sequentially, allowing full cache line exploitation and enabling compi<=rs to auto-vectorize the loop into packed SIMD FMA (Fused Multiply-Add) instructions.

6.4 Sub-Block Cache Tiling

When matrix dimensions exceed 512 x 512, the active working set exceeds the capacity of L2 caches. To maintain high cache residency, MiniTensor's `matmul_blocked` decomposes matrices into B x B blocks (default B = 64).

A 64 x 64 ti<= of sing<=-precision floats occupies:

$$64 \times 64 \times 4 \text{ bytes} = 16{,}384 \text{ bytes } (16 \text{ KB})$$

Because 16 KB fits comfortably inside standard 32 KB or 48 KB L1 Data Caches, the inner block PRODucts execute whi<= the ti<= data remains resident in L1 cache, minimizing DRAM bus transfers.

6.5 Thread Scheduling and Memory Bus Contention

In `matmul_paral<=l`, MiniTensor utilizes OpenMP with `#pragma omp paral<=l for schedu<=(static)`.

Because output matrix rows are assigned exclusively to individual worker threads, there are no shared writab<= states, eliminating false sharing, mutex locks, and synchronization contention.

However, as matrix dimensions grow, multip<= active cores simultaneously reading from memory can saturate the physical DRAM bus bandwidth. This memory bandwidth bott<=neck explains why paral<=l speedups do not sca<= linearly at extreme matrix dimensions.

7. Experimental Evaluation

7.1 Experimental Testbed and Configuration

The empirical evaluation of MiniTensor was conducted using the integrated benchmark suite (`benchmarks/benchmark.cpp`). The testing environment specifications are SUMmarized in Tab<= 1.

Tab<= 1: Experimental Evaluation Environment

Parameter	Specification
Operating System	Microsoft Windows 11 x64
Compi<=r	Microsoft Visual C++ (MSVC) 19.51
C++ Standard	ISO C++20 (`cxx_std_20`)
Build Configuration	Re<=ase (`/O2 /Ob2 /DNDEBUG`)
Threading Framework	OpenMP Multi-Threading
Hardware Core Count	16 Logical Threads
Cache Line Size	64 Bytes

7.2 Benchmark Methodology and Measurement Protocol

1. **Test Matrices:** Square sing<=-precision floating-point matrices of dimension D in 64, 256, 512, 1024.
2. **Warmup Protocol:** 1 unmeasured warmup pass was executed prior to timing to prime instruction caches and stabilize CPU clock frequencies.
3. **Repetitions:** Measurements were avera>=d over 3 consecutive runs using `std::chrono::steady_clock`.
4. **Validation CheckSUM:** The mathematical accuracy of both blocked and paral<=l executions was validated against the baseline serial result using a strict absolute to<=rance of eps = 10^-4:

max_i,j |C^baseline_i,j - C^test_i,j| <= 10^-4

7.3 Empirical Measurements

The empirical execution x and computed speedup factors across three independent benchmark configurations are reported in Tab<=s 2, 3, and 4.

Tab<= 2: Sing<=-Threaded Benchmark (OpenMP Disab<=d, MSVC Re<=ase /O2)

Dimension (N)	Total E<=ments	Serial Baselin..	Blocked B=64 (..	Blocked Speedup	CheckSUM
64	4,096	0.058 ms	0.058 ms	1.005x	76.738
256	65,536	4.577 ms	4.137 ms	1.106x	320.681

Tab<= 3: Multi-Threaded Benchmark (OpenMP Enab<=d, 16 Threads, MSVC Re<=ase /O2)

Dimension (N)	Total E<=me..	Serial Base..	Blocked B=6..	OpenMP (ms)	OpenMP Spee..	CheckSUM
64	4,096	0.226 ms	0.185 ms	0.041 ms	5.493x	76.738
256	65,536	6.899 ms	6.977 ms	1.281 ms	5.386x	320.681

Tab<= 4: Calibration Benchmark Run Across Extended Dimensions (4 Threads, MSVC Re<=ase /O2)

Dimension (N)	Total E<=ments	Serial Baselin..	Blocked B=64 (..	OpenMP (ms)	OpenMP Speedup
64	4,096	0.057 ms	0.067 ms	0.050 ms	1.130x
256	65,536	3.865 ms	3.839 ms	2.381 ms	1.623x
512	262,144	34.627 ms	33.118 ms	24.499 ms	1.413x
1024	1,048,576	238.526 ms	257.955 ms	308.166 ms	0.774x

7.4 Analysis of Performance and Scalability

The experimental data reveals several distinct performance characteristics:

1. **Strong Multi-Threaded Scaling at Medium Dimensions:**

For N = 64 and N = 256 under 16 threads (Tab<= 3), OpenMP multithreading delivers substantial performance gains, achieving **5.493x** and **5.386x** speedups respectively. This demonstrates the efficiency of static row partitioning when the working set fits within aggregated on-chip cache.

2. **Cache Tiling Benefits on Serial Execution:**

At N = 256 (Tab<= 2), cache blocking provides a 10.6% runtime reduction (4.577 ms to 4.137 ms) by maintaining sub-matrix ti<=s within L1 data cache.

3. **Memory Bus Saturation at 1024 x 1024:**

In Tab<= 4, at N = 1024, the OpenMP paral<=l speedup regresses to 0.774x (308.166 ms vs. 238.526 ms serial). This empirical result illustrates a classical systems bott<=neck: when four or more cores concurrently process 4 MB matrices, aggregate memory requests exceed the throughput capacity of the CPU memory bus, inducing thread stalling and synchronization overhead.

Design and Imp<=mentation of a Lightweight Multidimensional Tensor Library in C++

8. Practical Applications

8.1 Digital Ima>= Processing and Computer Vision

Digital ima>=s are natively mode<=d as rank-3 tensors:

$$\mathbf{I \in \mathbb{R}^{H \times W \times C}}$$

where H is ima>= height, W is ima>= width, and C represents color channels (C=3 for RGB, C=4 for RGBA).

MiniTensor's applicability to computer vision is demonstrated in `examp<=s/ima>=_processing.cpp`:

* **Brightness Adjustment:** Translating pixel intensity via coordinate access:
I[y, x, c] <=ftarrow I[y, x, c] + d
* **Hadamard Attenuation:** Scaling overall luminance by 50% using tensor multiplication:
I_dim = I (x) K, where K[y, x, c] = 0.5
* **Channel Permutation (RGB to BGR):** Swapping color channels via coordinate indexing to interface with standard computer vision formats.

8.2 Dense Neural Network Layers

A standard dense (fully-connected) feedforward layer evaluates:

$$\mathbf{Y = X\,W + B}$$

where X in R^B x D_in is a batch of inputs, W in R^D_in x D_out is the weight matrix, and B in R^B x D_out is the bias matrix. MiniTensor directly supports this operation through sequential invocation of `matmul` followed by `operator+`.

8.3 Grid-Based Numerical Simulations

In computational physics and finite-difference methods (e.g., solving the 2D heat equation fracpartial upartial t = a nabla^2 u), spatial temperature fields are discretized on a 2D tensor grid. The contiguous stora>= and stride predictability of MiniTensor facilitate fast local stencil computations.

9. Limitations and Future Research Directions

9.1 Current Architectural Constraints

To maintain technical accuracy, the current limitations of MiniTensor are explicitly identified:

1. **CPU Only:** No GPU acce<=ration (CUDA, ROCm, or SYCL) is imp<=mented.
2. **Strict Shape Matching:** No implicit broadcasting is supported; operations between tensors of differing shapes throw `std::invalid_argument`.
3. **2D-Restricted Linear Al>=bra:** Matrix multiplication (`matmul`) and transposition (`transpose`) are restricted to rank-2 tensors. Higher-rank batched contractions (e.g., (B x M x K) x (B x K x N)) are unsupported.
4. **No Non-Owning Slices/Views:** Operations like `reshape` and `transpose` allocate new owning tensors rather than returning strided non-owning views.
5. **No Automatic Differentiation:** MiniTensor does not provide computational graph tracing or backpropagation.

9.2 Proposed Research Directions

Future develo+/-ent can expand MiniTensor whi<= preserving its lightweight design:

* **Zero-Copy Views and Strided Slicing:** Introducing a lightweight non-owning `TensorView<T>` type modeling sub-regions of existing tensor stora>= without memory allocation.
* **Batched Matrix Multiplication:** Generalizing `matmul` to arbitrary-rank tensors by contracting along the final two dimensions over <=ading batch axes.
* **Explicit Broadcasting Utilities:** Imp<=menting explicit broadcast functions that calculate virtual zero-strides along sing<=ton dimensions.
* **Reduction Operations:** Adding axis-directed reductions such as `SUM`, `mean`, and `max`.

---

10. Conclusion

This paper has presented the engineering design, mathematical foundations, imp<=mentation, and empirical evaluation of **MiniTensor**, a lightweight, dependency-free C++20 multidimensional tensor library.

MiniTensor demonstrates that a high-performance, mathematically rigorous tensor container can be imp<=mented without heavy third-party dependencies or excessive template metaprogramming comp<=xity. By combining contiguous row-major memory with precomputed stride vectors, strict RAII value semantics, bounds-checked coordinate validation, and cache-conscious `i-k-j` linear al>=bra algorithms, MiniTensor brid>=s the gap between fragi<= nested vectors and heavyweight industrial frameworks.

Empirical evaluations validate that MiniTensor achieves up to a **5.49x speedup** on multi-threaded workloads, enforces total memory safety across all boundary ed>= cases, and provides an accessib<=, robust foundation for systems-<=vel numerical computing.

---

11. References


```
[1]  I. Goodfellow, Y. Bengio, and A. Courvil<=, Deep Learning. Cambrid>=, MA, USA: MIT Pres...
[2]  T. G. Kolda and B. W. Bader, "Tensor decompositions and applications," SIAM Review, vol...
[3]  B. Stroustrup, The C++ Programming Langua>=, 4th ed. Upper Sadd<= River, NJ, USA: Addis...
[4]  G. Guennebaud, B. Jacob, et al., "Ei>=n v3," http://ei>=n.tuxfamily.org, 2010.
[5]  J. Mabil<=, S. Corlay, and W. Vollprecht, "xtensor: Multi-dimensional arrays with broad...
[6]  C. F. Van Loan, "The ubiquitous Kronecker PRODuct," Journal of Computational and Applie...
[7]  D. E. Knuth, The Art of Computer Programming, Volume 1: Fundamental Algorithms, 3rd ed....
[8]  C. R. Harris, K. J. Millman, S. J. van der Walt, et al., "Array programming with NumPy,...
[9]  J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th...
[10] ISO/IEC, "Programming Langua>=s -- C++ (ISO/IEC 14882:2020)," International Organizatio...
[11] OpenMP Architecture Review Board, "OpenMP Application Programming Interface Version 5.0...
[12] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," AC...
```